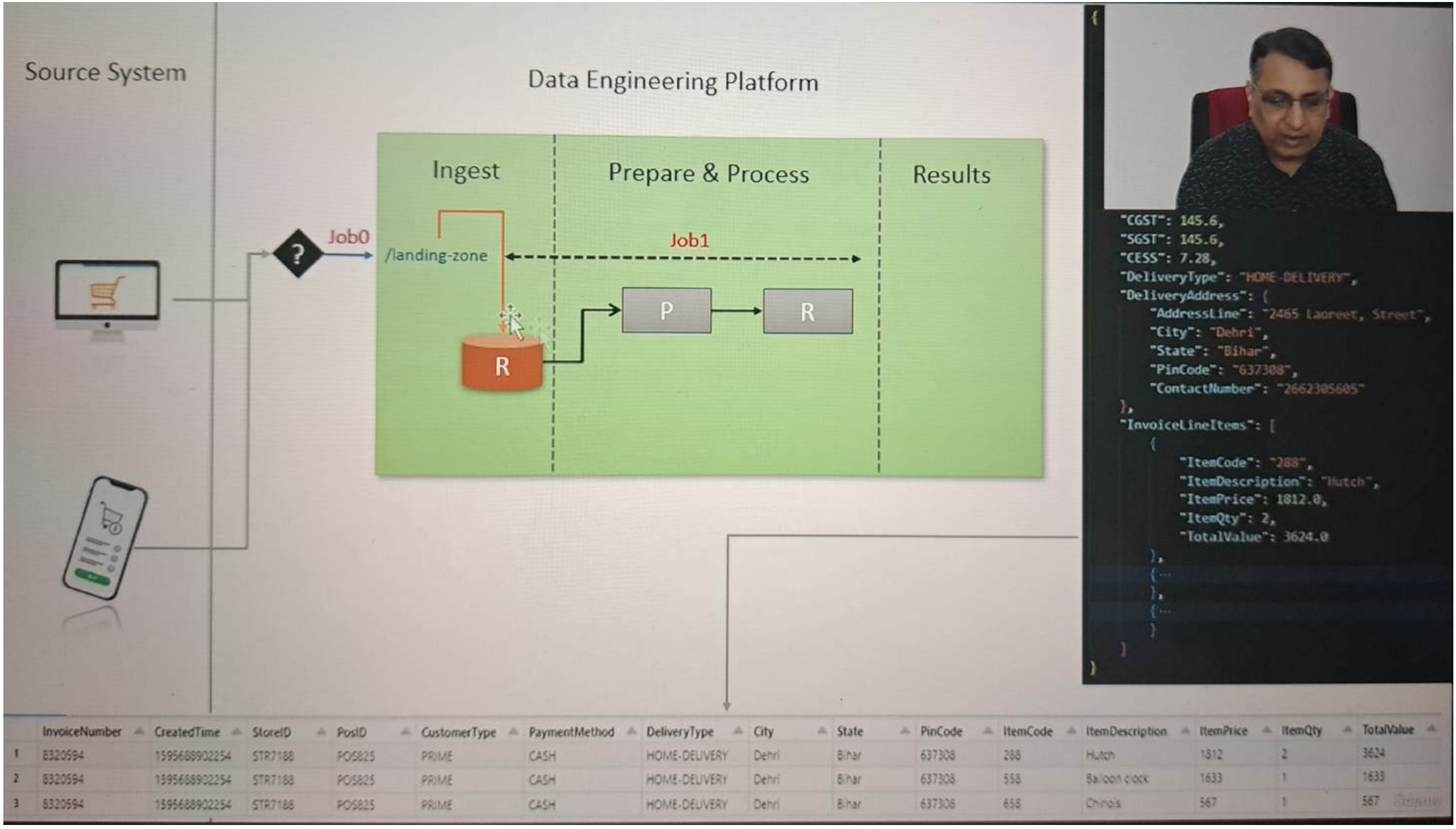

	InvoiceNumber	CreatedTime	StoreID	PosID	CustomerType	PaymentMethod	DeliveryType	City	State	PinCode	ItemCode	ItemDescription	ItemPrice	ItemQty	TotalValue
1	8320594	1595688902254	STR7188	POS825	PRIME	CASH	HOME-DELIVERY	Dehri	Bihar	637308	288	Hutch	1812	2	3624
2	8320594	1595688902254	STR7188	POS825	PRIME	CASH	HOME-DELIVERY	Dehri	Bihar	637308	558	Balloon clock	1633	1	1633
3	8320594	1595688902254	STR7188	POS825	PRIME	CASH	HOME-DELIVERY	Dehri	Bihar	637308	658	Chinois	567	1	567

```
                .drop("LineItem")
            )

    def appendInvoices(self, flattenedDF, trigger = "batch"):
        sQuery = (flattenedDF.writeStream
                .format("delta")
                .option("checkpointLocation", f"{self.base_data_dir}/chekpoint/invoices")
                .outputMode("append")
            )

        if (trigger == "batch"):
            return ( sQuery.trigger(availableNow = True)
                        .toTable("invoice_line_items"))
        else:
            return ( sQuery.trigger(processingTime = trigger)
                        .toTable("invoice_line_items"))

    def process(self):
        print(f"Starting Invoice Processing Stream...", end='')
        invoicesDF = self.readInvoices()
        explodedDF = self.explodeInvoices(invoicesDF)
        resultDF = self.flattenInvoices(explodedDF)
        sQuery = self.appendInvoices(resultDF)
        print("Done\n")
        return sQuery
```

```
                .drop("LineItem")
            )

    def appendInvoices(self, flattenedDF, trigger = "batch"):
        sQuery = (flattenedDF.writeStream
                    .format("delta")
                    .option("checkpointLocation", f"{self.base_data_dir}/chekpoint/invoices")
                    .outputMode("append")
                    .option("maxFilesPerTrigger", 1)
                )

        if (trigger == "batch"):
            return ( sQuery.trigger(availableNow = True)
                        .toTable("invoice_line_items"))
        else:
            return ( sQuery.trigger(processingTime = trigger)
                        .toTable("invoice_line_items"))

    def process(self):
        print(f"Starting Invoice Processing Stream...", end='')
        invoicesDF = self.readInvoices()
        explodedDF = self.explodeInvoices(invoicesDF)
        resultDF = self.flattenInvoices(explodedDF)
        sQuery = self.appendInvoices(resultDF)
        print("Done\n")
        return sQuery
```

05-streaming-batch Python

File Edit View Run Help Last edit was 14 minutes ago Provide feedback

Run all Connect Share Publish

```
        if (trigger == "batch"):
            return ( sQuery.trigger(availableNow = True)
                        .toTable("invoice_line_items"))
        else:
            return ( sQuery.trigger(processingTime = trigger)
                        .toTable("invoice_line_items"))

    def process(self, trigger = "batch"):
        print(f"Starting Invoice Processing Stream...", end='')
        invoicesDF = self.readInvoices()
        explodedDF = self.explodeInvoices(invoicesDF)
        resultDF = self.flattenInvoices(explodedDF)
        sQuery = self.appendInvoices(resultDF, trigger)
        print("Done\n")
        return sQuery
```

Shift+Enter to run
Shift+Ctrl+Enter to run selected text

```
    def runStreamTests(self):
        self.cleanTests()
        iStream = invoiceStreamBatch()
        streamQuery = iStream.process("30 seconds")

        print("Testing first iteration of invoice stream...")
        self.ingestData(1)
        self.waitForMicroBatch()
        self.assertResult(1249)
        print("Validation passed.\n")

        print("Testing second iteration of invoice stream...")
        self.ingestData(2)
        self.waitForMicroBatch()
        self.assertResult(2506)
        print("Validation passed.\n")

        print("Testing third iteration of invoice stream...")
        self.ingestData(3)
        self.waitForMicroBatch()
        self.assertResult(3990)
        print("Validation passed.\n")

        streamQuery.stop()
```

```
    def runBatchTests(self):
        self.cleanTests()
        iStream = invoiceStreamBatch()

        print("Testing first batch of invoice stream...")
        self.ingestData(1)
        self.ingestData(2)
        iStream.process("batch")
        self.waitForMicroBatch(30)
        self.assertResult(2506)
        print("Validation passed.\n")

        print("Testing second batch of invoice stream...")
        self.ingestData(3)
        iStream.process("batch")
        self.waitForMicroBatch(30)
        self.assertResult(3990)
        print("Validation passed.\n")
```

Cmd 3

```
isTS = invoiceStreamTestSuite()
isTS.runTests()
```

06-streaming-batch-test-suite Python

File Edit View Run Help Last edit was 9 minutes ago Provide feedback

Run all | Connect | Share | Publish

```
        self.ingestData(3)
        iStream.process("batch")
        self.waitForMicroBatch(30)
        self.assertResult(3990)
        print("Validation passed.\n")
```

Cmd 3

```
sbTS = streamingBatchTestSuite()
sbTS.runStreamTests()
```

Cmd 4

Python

```
sbTS.runBatchTests()
```

Shift+Enter to run
Shift+Ctrl+Enter to run selected text